

ReactJS

Easy Beginner's Guide

Function Components • JSX • Props • State • Events

1. Function Components

What is it?

A **Function Component** is simply a JavaScript function that returns some HTML-like code (JSX). It is the building block of every React app. Think of it like a custom HTML tag you create yourself.

Basic Example

```
function Greeting() { return <h1>Hello, World!</h1>; } // Use it like an HTML tag: // <Greeting />
```

Key Rules:

- Component name must start with a **Capital Letter** (e.g. MyCard, not mycard).
- It must **return** exactly one root element (wrap multiple elements in a <div> or <> </>).
- You can use the component anywhere like: <Greeting />

■ Tip: Every React app starts with a root component, usually called App.

2. JSX (JavaScript XML)

What is it?

JSX lets you write HTML-looking code inside JavaScript. React converts it to real JavaScript behind the scenes. It makes your UI code much easier to read and write.

Basic Example

```
function Profile() { const name = "Rahul"; const age = 25; return ( <div>  
<h2>Name: {name}</h2> <p>Age: {age}</p> <p>Adult? {age >= 18 ? "Yes" : "No"}</p>  
</div> ); }
```

JSX Important Rules:

- Use **{ }** curly braces to embed JavaScript expressions inside JSX.
- Use **className** instead of class (since class is a reserved JS keyword).
- Every tag must be closed — e.g. ,

- Return only ONE parent element. Use <> </> (Fragment) if you don't want an extra div.

■ Tip: JSX is not real HTML — it is JavaScript. React uses it to build the UI efficiently.

3. Props (Properties)

What is it?

Props are inputs you pass to a component — just like passing arguments to a function. They allow you to reuse the same component with different data.

Basic Example

```
// Define the component function WelcomeCard(props) { return ( <div>  
<h2>Welcome, {props.name}!</h2> <p>You are {props.age} years old.</p> </div> );  
} // Use it with different props function App() { return ( <div> <WelcomeCard  
name="Priya" age={22} /> <WelcomeCard name="Arjun" age={30} /> </div> ); }
```

With Destructuring (Cleaner Way)

```
function WelcomeCard({ name, age }) { return <h2>{name} is {age} years  
old.</h2>; }
```

Key Points:

- Props are **read-only** — a component cannot change its own props.
- You can pass strings, numbers, booleans, arrays, objects, or even functions as props.
- String props: name="Priya" | Number/Boolean props: age={22} isAdmin={true}

■ Tip: Props flow from Parent → Child only (one-way data flow).

4. State (useState Hook)

What is it?

State is data that can change over time inside a component. When state changes, React automatically re-renders the component to show the updated UI. We use the **useState** hook to add state.

Basic Example: Counter

```
import { useState } from 'react'; function Counter() { // count = current value,  
setCount = function to update it const [count, setCount] = useState(0); // 0 is  
the starting value return ( <div> <h2>Count: {count}</h2> <button onClick={() =>  
setCount(count + 1)}> Add +1 </button> <button onClick={() => setCount(0)}>  
Reset </button> </div> ); }
```

Another Example: Toggle

```
function LightSwitch() { const [isOn, setIsOn] = useState(false); return ( <div>  
<p>Light is {isOn ? "ON" : "OFF"}</p> <button onClick={() => setIsOn(!isOn)}>  
Toggle Light </button> </div> ); }
```

Key Points:

- **useState(initialValue)** returns [currentValue, setterFunction].
- Always use the setter function (setCount) — never modify state directly.
- Each component has its own independent state.

■ Tip: Props = data from parent (fixed). State = data inside component (can change).

5. Events

What is it?

Events in React work just like in regular HTML, but use camelCase names. You attach event handlers (functions) to elements to respond to user actions like clicks, typing, hovering, etc.

Common Events:

Event	React Name	Triggered When
Click	onClick	User clicks a button
Input Change	onChange	User types in an input field
Form Submit	onSubmit	User submits a form
Mouse Over	onMouseOver	Mouse hovers over element
Key Press	onKeyDown	User presses a keyboard key

Click Event Example

```
function ClickMe() { function handleClick() { alert("Button was clicked!"); }
return <button onClick={handleClick}>Click Me</button>; }
```

onChange - Input Field Example

```
import { useState } from 'react'; function NameInput() { const [name, setName] =
useState(""); function handleChange(event) { setName(event.target.value); // get
what user typed } return ( <div> <input type="text" placeholder="Enter your
name" onChange={handleChange} /> <p>Hello, {name}!</p> </div> ); }
```

Key Points:

- Pass the function **reference**, not a call: onClick={handleClick} ■ onClick={handleClick()} ■
- Use arrow functions for inline handlers: onClick={() => setCount(count + 1)}
- The event object (e or event) gives you details like e.target.value for inputs.

■ Tip: Combine Events + State to make interactive apps — that is the heart of React!

Quick Summary

Concept	What It Does	Key Hook/Syntax
Function Component	Defines a piece of UI	function Name() { return ... }

JSX	HTML inside JavaScript	{ } for JS expressions
Props	Pass data parent → child	function Card({ name }) { }
State	Data that changes in component	const [val, setVal] = useState()
Events	Handle user interactions	onClick, onChange, onSubmit

Happy Coding with React! ■